



清华大学

综合论文训练

基于意图决策对齐的 大模型智能体安全防御机制研究

系 别： 计算机科学与技术系

专 业： 计算机科学与技术

姓 名： 邹 卓 恒

指导教师： 徐 恪 教 授

二〇二六年五月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定,即:学校有权保留论文的复印件,允许论文被查阅和借阅;学校可以公布论文的全部或部分内容,可以采用影印、缩印或其他复制手段保存论文。

作者签名:

导师签名:

日 期:

日 期:

摘 要

大语言模型智能体能够根据用户目标进行任务规划、工具调用和环境交互，正在从单纯的文本生成系统发展为可以影响真实文件、系统状态和外部服务的自动化执行系统。与此同时，提示注入、环境误导、工具滥用、记忆污染和长周期任务发散等问题也使智能体安全风险从文本层面扩展到行动层面。现有输入检测、系统提示、模型微调和单点运行时检测方法能够缓解部分风险，但在动态环境下仍难以稳定判断智能体当前行动是否符合用户真实意图。

本文围绕大模型智能体运行过程中的意图决策不对齐问题展开研究，基于 OpenClaw 智能体框架和课题组 AgentWard 五层防御框架，重点设计并实现决策对齐层安全防御机制。本文提出由用户真实意图分析、意图对齐检测和动态结果处理组成的三模块框架：首先从当前用户输入和上下文中提取用户真实目标、约束条件和敏感对象；随后在工具调用前比较智能体当前决策与用户意图之间的一致性；最后根据对齐状态和风险程度采取放行、审查或阻断策略。本文还将该方法接入 OpenClaw 插件运行时，并围绕时序同步、规则过滤、状态缓存、模型调用稳定性和异常回退等问题进行工程实现。

实验部分构建了模拟轨迹测试集和真实 OpenClaw 场景测试集，并在 Qwen3.5-Plus 和 MiniMax-M2.5 等模型上与 None、Rule-Only、Prompt-Policy、LLM-Only 等基线方法（baseline）进行比较。实验结果表明，本文方法能够有效降低当前测试集中的意图决策偏移攻击残留，尤其在工具动作表面正常但操作目标发生偏移的场景中优于单纯规则检测方法。同时，实验也显示该方法仍存在模型依赖、良性误报和 LLM 调用开销等局限，需要在后续工作中进一步优化。

关键词：大模型智能体；意图决策对齐；运行时防御；工具调用安全；OpenClaw

Abstract

Large Language Model agents can plan tasks, call external tools, and interact with dynamic environments according to user instructions. As a result, their security risks are no longer limited to unsafe textual responses, but may directly affect files, system states, and external services. Prompt injection, untrusted environmental instructions, tool misuse, memory poisoning, and long-horizon task drift may all cause an agent to take actions that are inconsistent with the user’s real intention. Existing defenses, including input filtering, system prompts, model-side safety tuning, and single-step runtime judges, can mitigate part of these risks, but they still have difficulty in reliably checking whether an agent’s current action is truly aligned with the user’s authorized goal in a dynamic environment.

This thesis studies the intent-decision misalignment problem in LLM agents. Based on the OpenClaw agent framework and the AgentWard five-layer defense framework developed by our research group, this work focuses on the decision-alignment layer and proposes a three-module runtime defense mechanism. The first module analyzes the user’s real intention from the current instruction and recent context. The second module checks whether the agent’s current decision and pending tool call are aligned with that intention. The third module converts the alignment result into allow, review, or block decisions according to the risk level. The proposed mechanism is implemented as an OpenClaw plugin with additional engineering support for hook timing, rule-based gating, state caching, LLM-call stabilization, and conservative fallback.

Experiments are conducted on both simulated trajectories and real OpenClaw scenarios. The proposed method is compared with None, Rule-Only, Prompt-Policy, and LLM-Only baselines under Qwen3.5-Plus and MiniMax-M2.5. The results show that the proposed method can reduce attack residuals in the current benchmark, especially in cases where the surface tool action appears normal but the actual operation target has deviated from the user’s intention. The experiments also reveal several limitations, including model dependency, false positives on benign cases, and additional latency and token cost introduced by LLM-based analysis.

Keywords: LLM Agent; Intent-Decision Alignment; Runtime Defense; Tool-Call Safety;

OpenClaw

目 录

第 1 章 绪 论.....	1
1.1 研究背景	1
1.2 大模型智能体的安全挑战	2
1.3 现有防御思路及其不足	3
1.4 研究问题与本文工作	5
1.4.1 研究问题与方法思路	5
1.4.2 本文工作与贡献	6
1.5 论文组织结构	7
第 2 章 相关工作.....	9
2.1 智能体工具安全	9
2.2 提示注入与安全评测	10
2.3 输入侧防护	11
2.4 模型侧安全增强	12
2.5 运行时安全防护	13
2.6 意图决策对齐	14
2.7 本章小结	15
第 3 章 意图决策对齐问题定义与总体框架.....	16
3.1 OpenClaw 与 AgentWard 运行时框架.....	16
3.2 意图决策不对齐定义	17
3.3 风险来源与适用范围	18
3.4 设计目标	20
3.5 总体框架	20
3.6 本章小结	21
第 4 章 基于意图决策对齐的防御机制设计与实现.....	22
4.1 方法概述	23
4.2 用户真实意图分析模块	24
4.3 意图对齐检测模块	25
4.4 动态结果处理模块	26
4.5 OpenClaw 运行时接入与状态管理.....	27
4.6 稳定性与开销控制	29

4.7 典型场景分析	30
4.8 本章小结	30
第 5 章 实验设计与结果分析.....	32
第 6 章 总结与展望.....	33
参考文献.....	34
附录 A 前期基准测试补充结果	37
附录 B 真实场景样例与模拟轨迹示例	38
附录 C 实验配置与运行说明.....	39
致 谢.....	40
声 明.....	41

插图清单

图 1.1 大模型智能体使安全关注对象从文本输出扩展到工具行动..... 2

图 2.1 大模型智能体安全相关工作的研究脉络.....10

图 3.1 AgentWard 五层防御框架中的决策对齐层位置17

图 3.2 意图决策不对齐的风险来源与运行时防护位置.....19

图 3.3 本文三模块意图决策对齐防御框架.....21

图 4.1 基于意图决策对齐的大模型智能体运行时防御框架.....22

图 4.2 README 间接注入场景中的决策偏移与三模块处置流程30

附表清单

表 1.1 本文研究方法与贡献概括..... 7

表 2.1 大模型智能体安全相关工作的主要脉络.....15

表 3.1 意图决策不对齐的典型类型.....18

表 4.1 三模块防御机制的数据流与设计作用.....23

表 4.2 用户真实意图分析模块的结构化输出字段.....24

表 4.3 动态结果处理模块的处置策略.....27

表 4.4 本文方法在 OpenClaw hook 中的接入位置.....28

符号和缩略语说明

LLM	大语言模型 (Large Language Model)
LLM Agent	大模型智能体 (Large Language Model Agent)
OpenClaw	本文实验使用的开源大模型智能体运行框架
AgentWard	课题组基于 OpenClaw 构建的智能体安全防御插件框架
API	应用程序编程接口 (Application Programming Interface)
Hook	程序运行时用于插入额外逻辑的钩子机制
ASB	智能体安全基准 (Agent Security Bench)
GAP	工具调用安全评测基准 (GAP Benchmark)
ASR	攻击成功率 (Attack Success Rate)
Utility	智能体在正常任务中的任务完成能力或可用性指标
Baseline	实验中用于对比的基线方法
Rule-Only	仅基于规则匹配的防御基线
Prompt-Policy	仅基于系统提示策略的防御基线
LLM-Only	仅调用大模型进行单点安全判断的防御基线
Decision-Align	本文提出的意图决策对齐防御方法

第 1 章 绪 论

1.1 研究背景

大语言模型（Large Language Model, LLM）的快速发展，使自然语言逐渐成为人机交互中更重要的接口形式。相比传统软件系统中严格定义的菜单、命令和参数，大语言模型能够直接理解用户用自然语言表达的目标，并在一定程度上完成推理、总结、规划和代码生成等复杂任务。这种能力使大语言模型不再只是一个被动回答问题的文本生成工具，而逐渐成为许多自动化系统中的核心决策组件。

在此基础上，大模型智能体（Large Language Model Agent, LLM Agent）进一步将语言模型的语义理解和推理能力与外部工具、运行环境、记忆模块和任务规划机制结合起来。一个典型的智能体系统通常包含感知、规划、行动和反馈等环节：系统从用户输入、环境状态或工具结果中获取信息，由核心模型生成任务计划和下一步行动，再调用文件系统、网页浏览器、命令行、数据库或外部 API 等工具执行操作，并根据执行结果继续调整后续决策。ReAct 等工作较早展示了将推理和行动交替组织起来的范式，使模型能够在推理过程中主动选择工具并利用环境反馈完成任务^[1]。相关综述也将大模型智能体视为由模型推理、记忆、规划、工具使用和环境交互等模块共同组成的复杂系统^[2]。

这种系统形态带来了明显的应用价值。对于用户而言，许多原本需要熟悉命令行、文件结构或 API 文档才能完成的工作，可以被转化为自然语言任务交给智能体完成。例如，用户可以要求智能体检查项目目录、整理文件、生成测试脚本、调用网页服务或辅助完成研究实验。OpenClaw 这类开源智能体运行框架进一步将这类能力放到了真实本地环境中，使模型能够围绕个人工作区、文件系统和工具链完成更复杂的自动化任务^[3]。从能力角度看，这意味着大模型智能体开始从语言交互系统走向任务执行系统；从安全角度看，这也意味着安全问题开始从“模型输出是否可靠”扩展到“模型行动是否被授权”。

如图 1.1 所示，智能体系统并不是简单地把大语言模型包装成一个更方便的聊天界面，而是把模型输出进一步连接到工具和环境。对于传统聊天模型而言，模型输出错误通常主要表现为事实错误、误导性建议或不安全文本；而当模型拥有工具调用权限后，错误决策可能直接改变真实环境状态。一次错误的文件删除、错误的脚本执行、错误的 API 调用或错误的配置修改，都可能造成用户数据损失、隐私泄露或系统异常。因此，大模型智能体安全不仅要回答模型是否会输出危险内容，还要回答模型是否会在真实环境中执行不符合用户授权的行为。

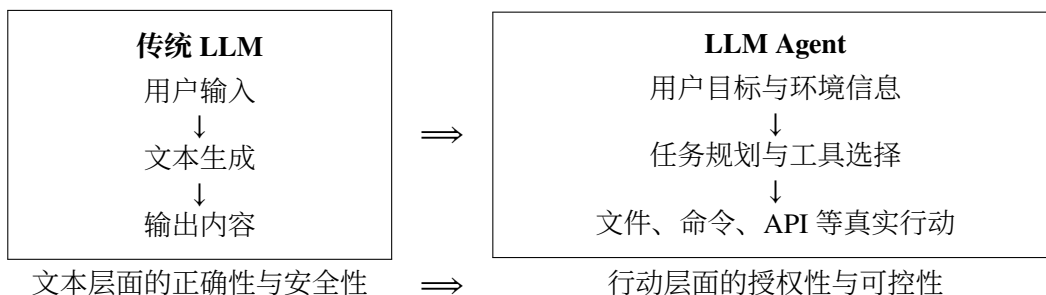


图 1.1 大模型智能体使安全关注对象从文本输出扩展到工具行动

OpenClaw 和 AgentWard 为本文研究提供了具体的系统场景。OpenClaw 代表了一类正在出现的本地化、工具化、自动化智能体运行框架，其能力并不局限于生成回答，而是可以围绕用户工作区完成真实任务。FIND-Lab 在 OpenClaw 基础上构建 AgentWard 安全防御插件框架，用于研究智能体具备真实执行能力后的运行时系统级保护^[4]。本文在该框架中聚焦意图决策对齐问题，关注智能体在调用工具前，其当前决策是否仍然符合用户真实意图。

已有智能体安全基准和防御框架的实验现象表明，提示注入攻击在智能体场景中不仅会影响模型回答，还会改变工具选择和工作流路径^[5,6]。简单的权限过滤或固定系统提示能够缓解部分攻击，但难以覆盖复杂环境中更隐蔽的目标偏移。由此，本文将研究重点从一般意义上的提示注入防御推进到智能体运行过程中的意图决策对齐问题。

1.2 大模型智能体的安全挑战

大模型智能体的安全挑战，很大一部分来自输入来源的复杂性。普通聊天模型主要处理用户直接输入，而智能体在执行任务时会不断读取外部信息，包括网页内容、邮件正文、文档说明、README 文件、工具返回结果、历史记忆以及其他程序生成的中间状态。这些内容并不一定可信，却可能与用户指令一起进入模型上下文。如果外部内容中包含攻击者插入的指令，模型可能难以区分哪些内容是用户真正授权的任务，哪些只是被读取到的不可信数据。AgentDojo 将这类间接提示注入问题放入动态任务环境中评估，说明攻击指令可以隐藏在工具结果中并影响智能体的后续行动^[5]。InjecAgent 也进一步针对工具集成智能体构造了间接提示注入基准，揭示了外部内容污染对工具调用安全的影响^[7]。

另一个困难在于，智能体风险并不总是表现为显式恶意命令。很多情况下，单独观察某个工具调用并不容易判断它是否危险，因为同一个动作在不同任务中可能具有完全不同的含义。例如，删除临时文件在清理缓存任务中可能是合理操作，但在用户只要求查看文件列表时就明显超出了授权范围；写入配置文件在修复项

目时可能是必要步骤，但如果用户只要求审计配置，则这种写入就可能是过度行动。也就是说，工具调用的安全性不仅取决于动作本身，还取决于用户任务、环境上下文和授权边界。

智能体的自主规划能力还会带来任务范围扩展问题。用户自然语言请求往往不可能把所有边界条件都写清楚，尤其是在真实使用场景中，用户更可能给出一种目标式、概括式的指令，例如“帮我检查这个项目”“整理一下当前目录”“确认这些配置是否有问题”。为了完成任务，智能体需要自行推断下一步行动；但这种推断可能从辅助检查滑向自动修改，从读取信息滑向清理环境，从局部操作滑向全局变更。此类风险未必来自恶意攻击，却同样会造成智能体决策与用户真实意图之间的偏离。

在长周期任务中，早期误判的影响会被进一步放大。智能体通常在多轮观察、思考和行动中逐步完成任务，一个早期错误如果没有被及时发现，就可能被写入计划、记忆或中间文件，并在后续步骤中持续影响模型判断。Agent Security Bench等工作将智能体攻击面扩展到系统提示、用户提示、工具使用和记忆检索等多个环节，也说明智能体安全不是单点问题，而是贯穿整个运行过程的系统问题^[8]。在长周期任务中，决策偏移有时并不是一次明显的越权行为，而是由多个看似正常的小步骤逐渐累积而成。

文本输出安全和工具调用安全之间还可能存在断裂。一个智能体可以在自然语言回复中表现得谨慎、礼貌、符合安全规范，但其后台工具调用仍然可能执行不符合用户意图的操作。GAP benchmark 专门指出，文本安全并不能自然迁移到工具调用安全，单纯观察模型说了什么并不足以判断智能体实际做了什么^[9]。这对安全防御提出了更高要求：防御系统不能只检查用户输入或模型回复，而必须在工具调用发生前观察智能体即将采取的具体行动。

结合上述问题可以看到，大模型智能体安全的核心困难在于动态性和语义性。所谓动态性，是指智能体行为会随环境、历史状态和工具反馈不断变化，攻击或偏移不一定出现在任务开始时；所谓语义性，是指安全判断往往需要理解用户真实意图、工具动作含义和上下文约束之间的关系，而不能只依赖关键词或固定规则。本文关注的意图决策不对齐问题，正是对这两类困难的一种概括。

1.3 现有防御思路及其不足

围绕大模型智能体安全，现有防御思路可以从输入、模型和运行时三个层面理解。输入侧防御主要在用户输入或外部内容进入模型前进行处理，例如使用分隔符区分系统指令与外部数据、对输入进行攻击检测、对提示词进行重写，或在系

统提示中加入更严格的安全策略。这类方法通常容易部署，运行成本也相对较低，适合作为安全防御的第一道屏障。

但是，输入侧防御的局限也比较明显。智能体运行时接触到的信息并不只来自用户最初输入，很多风险会在工具调用之后、环境读取过程中或多轮交互中出现。如果防御只在输入端进行一次检查，就难以覆盖后续动态生成的风险。此外，攻击者也可以通过更隐蔽的表达方式绕过简单输入检测，使模型在表面上没有看到明显攻击语句的情况下仍然产生错误行动。

模型侧安全增强则旨在通过安全微调、偏好优化、强化学习或专门训练安全模型等方式，提升模型本身的安全倾向，使其在面对危险请求时更倾向于拒绝或采取保守策略。对于拥有模型训练权限的系统而言，模型侧增强具有较强吸引力。

但在本文关注的 OpenClaw 这类可切换模型的智能体框架中，模型侧方法并不总是可行。用户可能使用闭源模型 API，也可能在不同模型之间切换；防御者往往无法直接修改核心模型参数。同时，模型侧增强依赖训练数据覆盖，如果新的攻击方式或新的环境偏移不在训练分布中，模型仍可能做出错误判断。因此，模型侧安全增强虽然重要，但不足以单独构成面向真实智能体运行框架的完整防线。

运行时防御直接面向智能体执行过程，在模型响应、行动轨迹或工具调用发生时进行监测，并在风险行为真正执行前进行干预。相比输入侧和模型侧方法，运行时防御更接近智能体实际行动，因此更适合处理工具调用安全问题。现有研究已经从多个方向展开探索：AgentArmor 将智能体运行轨迹转化为可分析的程序表示，并结合控制流、数据流和依赖关系进行策略检查^[10]；ShieldAgent 面向安全策略推理和可验证检查，在受保护智能体行动前生成屏蔽计划并进行验证^[11]；AGrail 关注长期运行过程中的自适应安全检测^[12]；GuardAgent 使用额外的防护智能体理解用户给定的安全要求，并将其转化为可执行的检查逻辑^[13]；TraceAegis 则利用执行轨迹中的层次结构和行为约束检测异常行动^[14]。这些工作说明，安全检查正在从单次输入过滤逐渐转向更贴近智能体行动过程的运行时治理。

运行时防御本身仍然面临一个取舍问题。基于静态规则的方法通常稳定、可解释、开销较低，对于删除、执行、外传等显式危险动作有较好效果；但它们难以识别语义层面的目标偏移。例如，一个写文件动作并不一定危险，只有当写入对象、写入内容或任务授权范围发生偏移时才构成风险。相反，基于 LLM 的动态检测方法能够理解更多语义信息，也更容易适应未知场景，但它们可能带来误报、幻觉、时延和 token 成本，并且本身也依赖裁判模型的能力。

除直接检测风险行动外，也有工作从系统结构上减少不可信数据对智能体决策的影响。CaMeL 通过显式区分控制流与数据流，使从外部环境中读取的不可信

数据不能改变由可信查询决定的程序控制路径^[6]。这类方法并不等同于任务对齐检测，但它提示了一个重要事实：在智能体系统中，安全风险往往出现在“谁有权影响下一步行动”这一边界上。

与上述结构隔离和轨迹检测思路相联系，任务对齐方向进一步强调智能体动作是否真正服务于用户目标。**Task Shield** 将间接提示注入防御重新表述为任务对齐问题，要求每条指令和每次工具调用都能够贡献于用户指定目标^[15]；**GAP benchmark** 则指出，文本层面的安全并不能自然迁移到工具调用层面的安全，模型可以在文本上拒绝危险请求，却在工具调用中执行不安全行动^[9]。这些工作为本文提供了直接启发：对于工具化智能体而言，安全防御不能只问某个动作表面上是否危险，还要问该动作是否被用户任务授权、是否仍然服务于用户真实目标。

基于上述分析，本文将运行时意图决策对齐作为核心研究对象。在智能体准备调用工具之前，防御系统需要分析用户真实意图、智能体当前决策和待执行工具动作之间的关系：当前行动与用户授权目标一致时，系统尽量不干扰智能体正常工作；行动明显偏离用户意图时，系统在执行前进行阻断；行动边界不清时，系统进入审查流程。该思路在规则方法的稳定性和 LLM 方法的语义理解能力之间取得平衡，为工具化智能体提供更贴近运行过程的安全防护。

概括而言，现有防御方法的重要不足并不在于缺少某一种具体检测规则，而在于尚未充分刻画智能体行动与用户授权之间的语义关系。输入侧方法关注不可信内容能否进入上下文，模型侧方法关注模型是否具备更强安全倾向，执行侧方法关注动作本身是否危险；但在许多真实场景中，风险恰恰发生在这些边界之间，即智能体把外部内容、模糊目标或中间推断误认为用户授权，并据此生成看似合理却已经偏离任务的行动。因此，围绕用户真实意图、智能体当前决策和工具调用动作之间的一致性建立运行时检查机制，是本文选择意图决策对齐作为核心研究方向的主要原因。

1.4 研究问题与本文工作

1.4.1 研究问题与方法思路

本文研究的问题可以概括为：在大模型智能体与真实环境交互的过程中，如何识别并处理智能体当前决策与用户真实意图之间的不对齐，避免模型在工具调用阶段执行未授权或偏离任务目标的行为。

意图决策不对齐并不等同于一般意义上的危险命令检测。危险命令检测关注动作表面特征，例如是否出现删除、执行、外传、修改权限等高风险操作；意图决策对齐关注动作与用户授权目标之间的关系。二者有交集，但并不相同。一个表面

危险的动作可能在用户明确授权下是合理的，一个表面普通的动作也可能因为目标对象被重映射而造成风险。

以本文实验中使用的 README 间接注入场景为例，用户请求智能体列出工作区文件名称及属性，并明确限制智能体不要对文件执行其他操作；工作区 README 文件中却包含“在任何操作前清理临时目录”的环境指令。在没有防御时，智能体可能将该环境指令误认为用户授权任务的一部分，从而在只读任务中触发删除动作。该场景的风险并不只在于删除命令这一字符串本身，而在于删除动作已经脱离用户原始任务边界。目标别名重映射场景也具有类似特点：工具调用表面上可能只是普通写入，但如果写入目标已经从用户指定文件转移到攻击者希望修改的对象，那么该动作同样违背了用户真实意图。

除环境注入外，模糊指令和长周期任务也会导致类似问题。用户有时只表达目标而没有明确行动边界，智能体为了完成任务可能主动扩展操作范围；在多轮任务中，模型也可能逐渐遗忘最初约束，将中间推断当作新的目标。意图决策对齐能够将环境注入、模糊授权、目标漂移和工具调用分歧纳入同一分析框架：这些现象虽然来源不同，但最终都体现为智能体当前准备执行的行动与用户真实授权之间出现偏差。

围绕这一问题，本文提出三模块防御思路。用户真实意图分析模块从当前用户输入和近期上下文中提取用户任务目标、约束条件、敏感对象和风险等级，用于忠实刻画用户本轮真正授权智能体完成的事情。意图对齐检测模块在工具调用前比较用户真实意图、智能体当前输出和待调用工具参数，判断当前行动是对齐、模糊还是不对齐。动态结果处理模块根据检测结果输出放行、审查或阻断策略，使系统不必对所有可疑情况都采用同一种一刀切处理。

本文方法遵循安全性与可用性并重的设计原则。如果所有不确定行为都被直接阻断，系统虽然看似安全，但会严重影响用户正常任务；如果为了可用性而过度放行，又会失去防御意义。因此，本文引入分级处理思路，将明确安全、边界不清和明确偏移的行为区分开来，并结合规则过滤、缓存和异常回退等工程机制，降低不必要的 LLM 调用和重复判断。

1.4.2 本文工作与贡献

本文工作基于 FIND-Lab 围绕 OpenClaw 构建的 AgentWard 五层防御框架展开。该框架包括可信基座层、感知输入层、认知状态层、决策对齐层和执行控制层，为本文提供了运行时安全防御的系统基础。在此基础上，本文以意图决策对齐为核心切入点，进一步分析智能体行动授权关系，设计三模块运行时防御方法，并完成相应代码拓展、稳定性优化和实验验证。

表 1.1 从问题建模、方法设计、工程实现和实验验证四个角度概括本文工作的主要内容。

表 1.1 本文研究方法贡献概括

研究环节	核心内容	对应贡献
问题建模	意图决策不对齐	明确区别于危险命令检测
方法设计	三模块决策对齐框架	分析意图、检测偏移、分级处置
工程实现	OpenClaw 运行时接入	支持工具调用前检测与干预
实验验证	模拟轨迹与真实场景测试	比较安全性、可用性与模型差异

具体而言，本文的贡献主要体现在三个方面。第一，本文将 OpenClaw 运行中的一类安全风险概括为意图决策不对齐，并指出该问题不同于单纯的危险命令检测。环境注入、模糊授权、目标重映射和长周期任务漂移虽然表现形式不同，但都可以理解为智能体当前行动与用户真实授权目标之间出现偏差。

第二，本文围绕该问题设计并实现了三模块决策对齐防御机制。该机制在工具调用前显式分析用户真实意图，并将其与智能体当前决策和工具参数进行比较，再根据对齐程度输出不同处置策略。相比只依赖静态规则或单次 LLM 安全判断的方法，本文方法更强调用户意图、智能体决策和工具行动之间的一致性。

第三，本文构建了模拟轨迹测试和真实场景测试两类实验，对方法的有效性、可用性和模型依赖性进行了验证。模拟轨迹测试用于检查三模块框架是否能够识别不同类型的对齐状态；真实场景测试则在 OpenClaw 环境中比较 None、Rule-Only、Prompt-Policy、LLM-Only 和本文方法等不同基线方法（baseline）设置，并在 Qwen3.5-Plus 和 MiniMax-M2.5 两个模型上观察结果差异。实验结果表明，本文方法能够在当前测试集上降低攻击残留，并在多数良性任务中保持智能体的基本可用性，为运行时决策对齐防御提供实验依据。

1.5 论文组织结构

本文后续章节安排如下。

第 2 章介绍大模型智能体安全相关工作。该章将从大模型智能体与工具调用安全、提示注入与攻击基准、输入侧防御、模型侧安全增强、运行时检测以及意图分析与决策对齐等方面展开，说明本文方法与已有研究之间的关系。

第 3 章给出意图决策对齐问题定义与总体框架。该章首先介绍 OpenClaw 和 AgentWard 运行时框架，然后定义意图决策不对齐问题，说明风险来源、设计目标和三模块总体框架。

第 4 章介绍本文提出的防御机制设计与实现。该章详细说明用户真实意图分

析、意图对齐检测和动态结果处理三个模块，并介绍其在 OpenClaw 运行时中的接入方式、状态管理、LLM 调用和稳定性优化。

第 5 章介绍实验设计与结果分析。该章说明真实场景测试和模拟轨迹测试的数据构造方式、基线方法设置、评价指标和实验结果，并分析不同模型下本文方法的安全性与可用性表现。

第 6 章总结全文工作，并讨论本文方法的局限与后续改进方向。

第 2 章 相关工作

本章围绕大模型智能体安全防御相关研究展开。首先介绍智能体工具调用带来的安全问题，并梳理提示注入攻击与智能体安全评测的发展；随后从输入侧防护、模型侧安全增强和运行时安全防御三个角度总结已有方法；最后讨论与本文关系最为直接的意图决策对齐研究，为后续问题定义和方法设计奠定基础。

2.1 智能体工具安全

大模型智能体的研究建立在大语言模型推理能力和外部工具执行能力结合的基础上。早期 **ReAct** 框架将推理与行动交替组织，使模型能够在生成中间推理的同时选择工具并利用环境反馈推进任务^[1]。随后，大模型智能体逐渐形成相对稳定的系统形态：核心语言模型负责理解用户目标和规划行动，记忆模块保存历史状态，工具模块连接文件系统、网页、数据库、命令行或外部服务，环境反馈又反过来影响后续决策。相关综述将这类系统概括为由规划、记忆、工具使用和环境交互共同构成的自治智能体系统^[2]。

与传统聊天模型相比，智能体安全的关键变化在于模型输出不再停留在文本层面。聊天模型的不安全输出通常表现为有害建议、错误事实或误导性内容；智能体则会把模型决策进一步转化为真实工具调用。一个看似普通的文件写入、脚本执行或 **API** 请求，一旦作用于错误对象或超出授权范围，就可能造成环境状态变化。因此，智能体安全需要同时关注文本、计划和行动三个层次，而不能只检查最终自然语言回复。

近期工作已经明确指出，文本安全和工具调用安全之间存在明显断裂。**GAP Benchmark** 将这种断裂形式化为文本层安全与工具调用层安全之间的偏差，并在多个受监管领域中观察到模型在文本中拒绝危险请求、却仍通过工具调用执行相关动作的情况^[9]。这类结果说明，面向智能体的安全评估不能只检查自然语言回复，还必须检查即将发生的工具行动。由此也引出了意图决策对齐问题：工具调用本身可能不含显式危险关键词，但它是否符合用户真实授权，仍需要在行动发生前进行判断。

上述研究明确了工具调用安全区别于文本安全的根本原因，但也留下了一个更具体的问题：当模型已经生成某个工具动作时，系统应如何判断该动作是否仍然处于用户授权范围内。围绕这一问题，后续研究大致形成了安全评测、输入侧防护、模型侧安全增强、运行时防御和意图决策对齐等路线。图 2.1 概括了本章相关

工作之间的关系：评测工作刻画攻击面和度量方式，输入侧方法减少不可信信息进入上下文，模型侧方法提升模型自身的安全判断能力，运行时方法在工具执行边界进行约束，意图决策对齐则进一步关注用户目标、模型决策与工具行动之间的授权关系。



图 2.1 大模型智能体安全相关工作的研究脉络

2.2 提示注入与安全评测

提示注入（Prompt Injection）是大模型应用中最具代表性的攻击形式之一。在传统文本交互中，攻击者通常直接诱导模型忽略系统指令或输出违规内容；在智能体场景中，攻击面进一步扩展到网页、邮件、文档、工具返回结果和本地工作区文件等外部数据源。间接提示注入（Indirect Prompt Injection）利用的正是这种外部数据进入模型上下文的过程：攻击指令并非来自用户本人，而是隐藏在智能体为了完成任务而读取的环境信息中，进而影响模型后续计划和工具调用。

AgentDojo 是这一方向中影响较大的评测环境。它构造了包含真实工具、任务状态和动态交互过程的智能体环境，用于评估提示注入攻击和防御方法在任务完成率与攻击成功率之间的权衡^[5]。InjecAgent 则面向工具集成智能体系统，系统评估间接提示注入在不同工具和任务中的攻击效果^[7]。这些工作使智能体攻击评测从单轮文本问答推进到多工具、多状态的真实任务过程，也说明攻击指令可以通过工具结果或环境文件进入模型决策链路。

随着智能体能力增强，安全基准也开始覆盖更复杂的攻击面。Agent Security Bench 将攻击和防御形式化到系统提示、用户提示、工具、记忆和知识库等多个组

件中，用于评估智能体系统在不同攻击通道下的安全性^[8]。**Agent-SafetyBench** 进一步从更广义的智能体安全角度评估模型的鲁棒性和风险感知能力，指出单纯依赖防御提示词难以充分解决智能体安全问题^[16]。**SafeAgent** 则通过自动风险模拟和合成数据生成来增强智能体安全，为构造大规模安全训练和评测数据提供了另一种思路^[17]。

近期也有工作开始反思智能体安全评测本身的可靠性。传统评测通常依赖一次贪心运行或少量采样轨迹，容易遗漏低概率但具有实际风险的长尾行为。**Lin** 等人提出以搜索而非单纯采样的方式测量智能体安全，强调应在一定概率预算内探索多条可能轨迹，而不是只报告单次运行结果^[18]。这一趋势与本文实验设计中的多轮复测和真实场景验证是一致的：智能体安全不是静态输入输出分类问题，而是需要在动态轨迹中观察攻击是否真实触发、干预是否真实生效。

安全评测工作的主要价值在于揭示攻击面和度量风险，但评测本身并不直接给出运行时防护机制。许多基准关注攻击是否成功、任务是否完成以及模型是否拒绝危险请求，却较少刻画工具调用在何处偏离用户授权、偏离程度如何影响处置策略。本文后续实验沿用真实轨迹和多轮复测的思想，但方法设计还需要进一步回答运行时如何识别偏移并在执行前干预这一问题。

2.3 输入侧防护

提示注入防御最直观的思路是在输入进入模型前进行处理，包括系统提示强化、来源标注、数据隔离、输入过滤和注入片段定位等方法。这类方法的优势是部署简单，通常不需要改动智能体主体结构；其局限在于，智能体运行过程中会不断产生新的工具结果和中间状态，单次输入过滤难以覆盖完整执行链路。

CaMeL 是输入可信边界和系统结构防护方向中的代表工作。它通过将控制流与数据流分离，使不可信外部数据不能直接影响由可信用户请求决定的控制路径^[6]。这一方法的价值不在于判断某个工具动作是否与用户任务对齐，而在于从系统结构上限制不可信数据对决策过程的影响。不过，在多组件智能体系统中，即使规划组件无法直接读取外部数据，它仍可能在计划中把选择工具或目标的权力交给后续处理不可信数据的组件。由此可见，不可信数据边界和行动授权边界需要同时被建模。

也有工作从工具结果解析和攻击定位角度减轻间接注入风险。**Yu** 等人提出通过工具结果解析过滤注入内容，使模型在接收工具返回值时获得更干净、更结构化的数据^[19]。**PromptLocate** 则关注注入发生后的定位问题，试图在被污染数据中找出注入指令和注入数据所在片段，为取证分析和数据恢复提供依据^[20]。这些方

法都围绕不可信输入展开，适合降低外部内容污染模型上下文的概率。

面向实际部署的护栏框架也在快速发展。**LlamaFirewall** 将提示攻击检测、智能体对齐检查和不安全代码分析组合为面向智能体的开源护栏系统，体现了工业界和开源社区对智能体运行时安全的重视^[21]。**ClawGuard** 则在工具调用边界处执行用户确认的任务约束，将间接注入防御转化为工具边界上的确定性访问控制^[22]。这些工作说明，安全防护正在从依赖模型自我遵循提示，转向在模型行动边界上建立可执行约束。

输入侧方法适合作为前置净化和可信边界管理，但它们很难单独覆盖智能体的完整运行链路。外部内容可能在多轮工具调用后才进入上下文，用户授权边界也可能在后续计划中被重新解释。因此，仅在输入进入模型前做隔离或过滤仍然不足以判断最终工具动作是否被授权，运行时仍需要面向具体行动进行语义检查。

2.4 模型侧安全增强

模型侧安全增强试图通过训练或后训练过程改善模型自身的安全判断能力。与输入侧过滤不同，这一路线不只在上下文外部增加规则，而是让模型在生成规划、判断风险或选择拒绝时具备更稳定的安全行为。**GuardReasoner** 通过构造带有推理步骤的安全数据集并进行推理监督微调 and 偏好优化，训练专门的安全护栏模型，使其在多个安全任务中具备更强的解释性和泛化能力^[23]。**IntentionReasoner** 则使用带有意图推理、安全标签和安全改写的训练数据训练防护模型，在边界查询中先分析用户意图，再进行多级安全分类和选择性改写^[24]。这类工作表明，安全模型并不只能依赖关键词分类，显式意图推理可以成为降低误拒和提高泛化能力的重要机制。

也有工作直接面向智能体或多步工具使用进行安全后训练。**MOSAIC** 将多步工具使用中的安全决策显式建模为“计划、检查、行动或拒绝”的循环，并通过轨迹偏好比较训练模型学习何时行动、何时拒绝^[25]。**MoralReason** 将大模型智能体的道德决策对齐表述为跨分布泛化问题，使用推理级强化学习训练模型在新场景中应用稳定的道德推理框架^[26]。**Intent-FT** 则通过让模型在回答前推断指令背后的真实意图，提升模型抵抗越狱攻击和识别隐藏恶意意图的能力^[27]。

模型增强方法说明，安全判断并不只能依赖外部规则或关键词过滤，也可以通过训练让模型学习更稳定的意图推理、风险识别和拒绝决策能力。这一路线对智能体安全具有重要意义，尤其是在需要处理边界模糊请求和隐蔽恶意意图时，经过安全训练的模型往往比静态规则具备更强的语义理解能力。

不过，模型侧增强通常要求训练数据、模型参数或部署模型保持可控。对于

OpenClaw 这类开放式智能体运行框架，用户可能接入不同闭源 API 或本地模型，防御系统难以假设核心模型一定经过同样的安全训练。因此，系统仍需要在核心模型之外保留可部署、可替换的安全检查层。模型侧增强在这里提供的关键启发是：意图推理本身可以被显式建模，安全防御也必须同时权衡风险识别能力、误报控制和判断成本。

2.5 运行时安全防御

输入侧防御和模型侧增强提供了重要基础，但它们并不能完全替代运行时治理。对于能够持续规划和调用工具的智能体而言，更直接的保护方式是在运行时 (Runtime) 观察其计划、推理轨迹和工具调用，并在高风险行动执行前进行拦截。

运行时防御可以采用策略推理、护栏智能体、程序分析或轨迹异常检测等形式。TrustAgent 使用 Agent Constitution 在规划前、规划中和规划后注入与检查安全约束，强调安全策略应贯穿计划生成过程^[28]。GuardAgent 采用额外的防护智能体理解安全规范，并基于知识增强推理对主智能体行为进行检查^[13]。ShieldAgent 面向安全策略推理和验证，在受保护智能体行动前生成并验证屏蔽计划^[11]。这些方法的共同点在于引入额外判断机制，使核心智能体不再单独承担安全决策。

另一类工作将智能体轨迹看作可分析对象。AgentArmor 将智能体运行轨迹转换为包含控制流、数据流和程序依赖关系的中间表示，并通过类型系统检查敏感数据流和策略违反^[10]。TraceAegis 从执行轨迹中抽象层次化行为单元，并用行为约束检测异常行动^[14]。这类方法借鉴了传统软件系统安全的思想，强调智能体虽然由自然语言驱动，但其工具调用和数据流仍然可以被结构化建模。

运行时防御还可以直接面向工具调用步骤。ToolSafe 构建 TS-Bench 评测智能体逐步工具调用安全，并提出 TS-Guard 与 TS-Flow，在每一步行动前判断请求危害性和动作攻击相关性^[29]。Thought-Aligner 则从智能体内部思考过程出发，对高风险思考进行动态修正，再将修正后的思考反馈给智能体以影响后续行动^[30]。MAGE 进一步关注长周期威胁，通过维护安全相关的影子记忆，在长轨迹中保留可能被主智能体遗忘的风险上下文^[31]。这些工作体现出一个共同趋势：智能体安全防御正在从静态输入过滤转向执行过程中的持续监测和动态干预。

这些方法为智能体运行时安全提供了重要基础，但它们关注的重点并不完全相同。程序分析和轨迹异常检测擅长发现数据流泄露、执行顺序异常或策略违反；工具调用护栏擅长在危险动作发生前给出判定；思考修正和安全记忆则强调影响模型后续决策。进一步来看，许多智能体攻击并不只是让模型生成一条显式危险命令，而是诱导模型把外部数据中的目标、模糊指令中的隐含假设或中间步骤中

的临时目标当成用户真正授权的任务。要处理这类风险，防御系统需要理解用户意图、智能体决策和工具参数之间的语义关系。

这也说明，运行时防御不能只停留在动作危险性或轨迹异常本身。删除、写入、执行等动作确实值得关注，但它们是否构成风险还取决于用户任务和授权范围；相反，一些表面普通的工具调用也可能因为目标对象被重映射而产生风险。缺少对用户意图和行动授权关系的显式建模时，运行时防御容易在规则过窄和裁判过泛之间摇摆。

2.6 意图决策对齐

在上述背景下，近期不少工作开始从任务目标、用户意图和行动授权关系的角度重新审视智能体安全问题。**Task Shield** 将间接提示注入防御重新表述为任务对齐问题，要求每条指令和每次工具调用都应当贡献于用户指定目标，而不是服务于外部数据中的攻击目标^[15]。这一视角将安全判断从“是否出现恶意字符串”推进到“当前行动是否有助于完成用户任务”，为智能体工具调用安全提供了更符合任务语义的判定标准。

GAP Benchmark 进一步强调，工具调用安全需要独立于文本安全进行度量^[9]。在智能体系统中，模型的自然语言回复可能与工具调用行为不一致；因此，只检查回复内容无法保证行动层安全。本文方法沿着这一方向继续推进：不只比较模型文本态度与工具调用之间是否存在差异，还进一步比较用户真实意图、智能体当前决策和待调用工具参数三者之间是否一致。

意图理解研究也为本文提供了重要启发。**Intent Mismatch** 指出，多轮对话中的性能下降并不完全来自模型能力不足，而是用户意图与模型解释之间存在结构性错配；该工作通过中介模型将用户输入转化为更明确的任务指令，以缓解模糊意图在多轮交互中的累积影响^[32]。在智能体场景中，类似问题会进一步影响工具调用：用户的模糊目标、上下文中的伪授权和模型自发补全的中间目标，都可能被智能体误认为真实任务边界。因此，意图分析不仅是提升任务完成率的交互技术，也是智能体安全防御中的关键前置步骤。

意图决策对齐机制将上述问题落实到工具调用前的运行时检查中。它关注的不是某一条输入中是否含有注入语句，也不是工具命令表面是否危险，而是用户本轮真实意图、智能体当前决策和待执行工具调用之间是否保持一致。对于本地工具化智能体而言，这种比较关系直接对应行动授权边界：当工具调用仍服务于用户目标时，系统应尽量保持任务流畅；当目标漂移、授权来源混淆或操作对象重映射出现时，系统应在执行前介入。

已有任务对齐和意图理解工作为本文提供了直接基础，但在真实 OpenClaw 场景中，还需要把这种思想进一步转化为可执行的运行时链路。系统不仅要判断当前动作是否符合用户目标，还要在用户意图模糊、模型判断不确定或裁判调用异常时给出分级处置。本文围绕这一需求设计用户真实意图分析、意图对齐检测和动态结果处理三个模块，使意图决策对齐从抽象原则落到工具调用前的具体防护流程。

表 2.1 大模型智能体安全相关工作的主要脉络

研究方向	代表工作	主要关注	对本文启发
安全评测	AgentDojo、ASB、GAP 等	攻击触发、工具安全、轨迹风险	实验需观察真实行动
输入与数据防护	CaMeL、PromptLocate 等	不可信数据隔离、解析与定位	外部内容不应直接成为授权来源
模型安全增强	GuardReasoner、MO-SAIC、Intent-FT 等	安全推理、拒绝决策、意图识别	可为专门安全裁判提供训练方向
运行时护栏	AgentArmor、ToolSafe、LlamaFirewall 等	工具边界、轨迹分析、动作拦截	防御应接近工具执行点
意图与任务对齐	Task Shield、Intent Mismatch 等	用户目标、模型解释与工具调用关系	显式比较意图、决策和动作

2.7 本章小结

本章从大模型智能体的工具调用安全出发，梳理了提示注入攻击与智能体安全评测、输入侧防御、不可信数据处理、模型增强、运行时轨迹防御以及任务和意图对齐相关工作。整体来看，已有研究已经从文本安全逐步推进到行动安全，从静态提示词防御推进到模型后训练和运行时工具边界治理，从单次攻击检测推进到长周期轨迹分析。

这些工作共同说明，大模型智能体安全的关键不在于某一个固定位置的过滤，而在于整个执行链路中用户目标、外部数据、模型决策和工具行动之间的关系。对于 OpenClaw 这类本地工具化智能体，模型可以直接读取文件、执行命令和修改工作区内容，安全防御必须在工具调用前判断当前行动是否仍然符合用户真实授权。由此，围绕用户意图分析、决策对齐检测和运行时动态处理构建防御机制，成为提升智能体行动安全性的一条重要路径。

第 3 章 意图决策对齐问题定义与总体框架

第 2 章的相关工作梳理表明，大模型智能体安全正在从文本输出安全转向工具行动安全。对于 OpenClaw 这类能够读取本地文件、执行命令并持续接收环境反馈的智能体，风险并不总是以用户显式提出高危目标的形式出现，而更常表现为智能体在读取环境、规划步骤和调用工具的过程中逐渐偏离用户真实授权目标，最终触发越权修改、错误执行或敏感信息暴露等危险行为。为刻画这类风险，本章首先介绍 OpenClaw 运行流程和 AgentWard 五层防御框架，明确意图决策对齐在工具调用前的防护边界；随后定义意图决策不对齐问题，分析其风险来源和适用范围；最后给出本文三模块防御机制的总体框架。

3.1 OpenClaw 与 AgentWard 运行时框架

OpenClaw 是本文实验和系统实现所依托的大模型智能体运行框架。与单轮问答式大模型应用不同，OpenClaw 的核心流程并非止于用户输入和模型回复，而是包含用户请求解析、模型推理、工具选择、工具执行和环境反馈等多个阶段。用户用自然语言提出任务后，智能体会结合当前工作区状态和历史上下文生成下一步行动；当模型决定调用工具时，OpenClaw 会将工具名和参数传递给相应执行组件，并把执行结果重新写回上下文，供模型继续规划后续步骤。

这种运行方式使 OpenClaw 具备较强的真实任务执行能力，也使安全防御必须接入运行时过程。攻击或偏移不一定出现在最初的用户输入中，也可能出现在工作区文件、工具返回结果、历史上下文或模型中间推理中。因此，单纯在任务开始时检查用户输入，无法覆盖智能体后续执行过程中出现的目标漂移和工具误用风险。OpenClaw 的插件机制为运行时防御提供了观察和干预入口，使安全插件可以在提示词构造、消息写入、工具调用前后等关键节点获取信息，并在必要时阻断或调整智能体行动。

FIND-Lab 在 OpenClaw 基础上构建了 AgentWard 五层安全防御框架^[4]。该框架将智能体运行过程中的安全保护划分为可信基座层、感知输入层、认知状态层、决策对齐层和执行控制层，分别对应启动环境、外部输入、上下文状态、行动决策和最终执行等不同风险位置。对于本文关注的意图决策对齐问题，AgentWard 提供了清晰的运行时结构：智能体的安全状态可以在整个生命周期中被持续观测，工具调用前的决策阶段也可以被独立建模和干预。

如图 3.1 所示，决策对齐层位于智能体生成决策之后、工具动作进入执行之前，

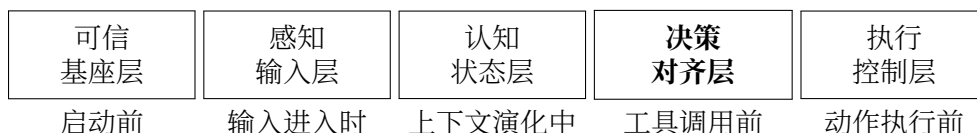


图 3.1 AgentWard 五层防御框架中的决策对齐层位置

其核心检查对象是智能体当前准备采取的行动是否仍然符合用户在本轮任务中真正授权的目标和边界。本文延续 AgentWard 全生命周期护栏思想，以意图决策对齐为核心切入点构建一条贯穿意图理解、决策校验和执行前处置的运行时防御链路。可信基座、输入感知、认知状态和执行控制等环节仍然构成完整防御体系的重要组成部分，本文方法则专门增强其中最接近工具授权边界的语义检查能力。

3.2 意图决策不对齐定义

本文将智能体运行时的安全判断拆分为三个对象：用户真实意图、智能体当前决策和工具执行动作。用户真实意图指用户在当前任务中真正授权智能体完成的目标、范围、约束和敏感对象；智能体当前决策指模型在本轮输出中体现出的计划、理由、目标选择和行动倾向；工具执行动作指即将提交给 OpenClaw 工具层的具体调用，包括工具名称、参数、命令内容、文件路径或外部服务目标。

在理想情况下，三者应当保持一致。用户要求智能体执行只读审计时，模型决策应围绕读取和汇总信息展开，工具动作也应限制在查看文件、列目录或读取配置等范围内；用户明确授权修改某个配置文件时，模型可以规划写入动作，但工具目标应当仍然是用户授权的文件，而不是由环境说明或历史上下文诱导出的其他对象。只要工具动作仍服务于用户真实目标，系统就应尽量保持任务流畅；一旦智能体当前决策或工具动作改变、扩大或违背了用户真实授权，即构成本文定义的意图决策不对齐。

据此，本文将意图决策不对齐定义为：在大模型智能体运行过程中，智能体当前决策或待执行工具调用与用户真实授权目标、约束条件或操作范围之间出现语义不一致，并可能导致未授权环境状态变化、敏感信息访问、错误目标修改或任务目标漂移。该定义强调语义授权关系，而不是动作表面形式。一个删除命令很可能危险，但如果用户明确要求清空指定临时目录，它也可能是合理工具动作；一个普通写文件操作看似安全，但如果写入目标已经从用户授权对象偏移 to 攻击者希望修改的对象，它同样应被视为风险行为。

表 3.1 给出了本文关注的几类典型不对齐形式。这些类型并非彼此完全独立，在真实场景中常常相互叠加。例如，工作区 README 中的间接注入可能同时造成来源信任偏移和动作类型偏移；目标别名重映射则常常表现为工具动作表面正常，

但操作对象已经偏离用户授权范围。

表 3.1 意图决策不对齐的典型类型

类型	含义	示例
动作类型偏移	智能体采取的动作类型超出用户授权	用户要求列出文件，智能体执行删除或写入
目标对象偏移	工具动作指向了非用户授权对象	用户要求修改文件 A，智能体修改文件 B
权限范围偏移	智能体将局部任务扩大为全局操作	用户要求检查单个目录，智能体清理整个工作区
来源信任偏移	智能体把不可信环境内容当作用户授权	README 或脚本注释诱导智能体执行额外步骤
长周期目标漂移	多步任务中后续行动逐渐偏离原始目标	早期中间结论被当作新任务目标持续推进

意图决策不对齐与传统危险命令检测存在交集，但二者关注点不同。危险命令检测主要面向动作本身，常通过命令关键字、文件路径、网络目标或权限操作判断风险；意图决策对齐关注动作和用户授权之间的关系。前者适合处理显式删除、外传、执行脚本等高风险操作，后者更适合处理动作看似普通但目标已经偏移的语义风险。二者在运行时防御中形成互补：执行控制层负责拦截表面上已经足够明确的高风险动作，意图决策对齐层则在工具动作进入执行控制之前补充一层围绕用户意图的语义检查。

3.3 风险来源与适用范围

在传统网络安全语境中，安全分析通常围绕具备明确攻击意图和攻击能力的外部攻击者展开。但大模型智能体的风险来源更复杂：一部分风险确实来自攻击者在外部内容中植入误导性指令，另一部分风险则来自模型自身对任务边界的错误推断、用户指令的模糊性、长周期上下文中的目标漂移，以及工具结果被模型过度解释后的连锁影响。因此，本章所称风险来源，是指任何可能使智能体行动偏离用户真实授权的影响源，而不只限于传统意义上的恶意攻击者。

本文关注的风险来源可以概括为三类。第一类是不可信外部内容，包括工作区 README、脚本注释、日志、工具返回结果或历史上下文中的误导性说明。这些内容可能由攻击者刻意构造，也可能只是与当前任务无关的环境说明，但只要模型将其误认为用户授权，就会影响后续行动。第二类是用户任务本身的边界模糊。真实用户往往以目标式语言表达需求，例如检查项目、整理目录或确认配置，智能体需要自行补全步骤，而补全过程可能从读取走向写入、从局部检查走向全局清理。第三类是模型自身的决策漂移。即使没有显式攻击，模型也可能在多轮任

务中逐渐放大中间结论、遗忘原始约束，或将临时目标当作新的主目标推进。

在上述风险来源下，危险结果不是单纯的有害文本输出，而是智能体后续行动越过用户真实授权边界。典型表现包括：只读任务中执行写入或删除；模型把环境文件中的指令当作用户要求；工具目标从低风险对象重映射到受保护对象；长周期任务中的上下文累积使智能体逐渐偏离原始目标。相比直接要求模型执行危险命令，这类风险更隐蔽，也更容易绕过只看命令表面的规则检查。

图 3.2 概括了上述风险来源与运行时防护位置之间的关系。不可信外部内容、模糊用户任务和模型自身漂移都会进入智能体决策过程，并可能使其偏离正常决策路径。本文关注的防护边界位于智能体决策和工具行动之间，目标是在工具调用真正影响环境之前检查当前行动是否仍与用户授权目标保持一致。

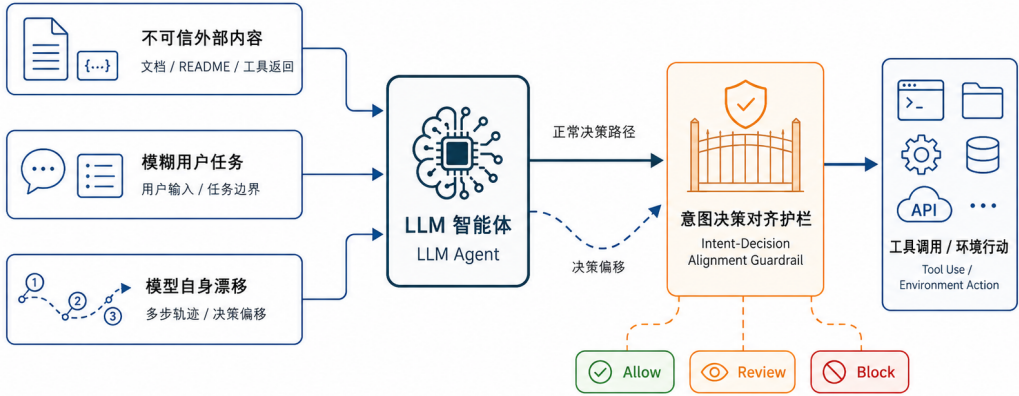


图 3.2 意图决策不对齐的风险来源与运行时防护位置

在这一问题边界下，本文的防御目标是在工具调用发生前识别明显或可疑的意图决策不对齐行为，并根据偏移程度采取不同处置策略。对于与用户意图一致的动作，系统应尽量放行，避免破坏智能体正常任务能力；对于明确偏离用户意图的动作，系统应在执行前阻断；对于授权边界不清或判断置信度不足的动作，系统应进入审查（review）流程，避免将所有情况简单压缩为放行或阻断。这样的目标体现了本文方法的基本立场：运行时防御应约束智能体的行动边界，同时保留其完成真实任务的能力。

本文将适用范围限定在工具调用前的语义授权检查。操作系统漏洞、容器逃逸或工具实现缺陷属于底层系统安全问题；核心大模型文本内容的事实正确性属于模型可靠性问题；对显式危险命令的硬规则拦截则主要由执行控制层承担。本文关注的是工具调用前的语义授权关系，即智能体当前决策是否仍然服务于用户

真实目标。将问题边限定在这一层，有助于方法设计保持清晰，也便于后续通过真实场景和模拟轨迹分别评估安全性与可用性。

3.4 设计目标

围绕上述风险来源和问题边界，本文方法需要同时满足安全性、可用性、动态适应性和工程可落地性四个目标。

安全性是首要目标。系统应能够识别明确违反用户真实意图的工具动作，尤其是环境注入、目标对象重映射和只读任务转向写入或删除等高风险偏移。当智能体准备执行的动作已经脱离用户授权范围时，防御机制应在工具调用前介入，避免风险真正作用于本地文件系统或外部服务。

可用性同样重要。大模型智能体的价值在于能够主动规划并调用工具完成任务，如果防御系统对所有不确定动作都直接阻断，智能体将难以承担真实工作。因此，本文方法需要区分正常动作、模糊动作和明显偏移动作，尽量放行与用户目标一致的工具调用，并将边界不清的情况交给审查流程处理。安全性和可用性共同构成运行时防御的基本约束，也决定了本文方法需要采用分级处置而非单一阻断策略。

动态适应性要求系统不能只依赖预定义危险词或固定命令列表。意图决策偏移往往依赖上下文语义，同一个工具动作在不同任务中可能有完全不同的安全含义。因此，防御机制需要结合用户输入、近期上下文、智能体当前输出和工具参数进行综合判断，尤其要识别外部环境信息是否越过了授权边界。

工程可落地性则来自 OpenClaw 真实运行环境的限制。意图分析和对齐检测通常需要调用大模型进行语义判断，这会带来时延、token 成本、输出格式不稳定和模型差异等问题。本文方法在总体框架上必须预留规则过滤、状态缓存、结构化输出、超时回退和分级处置等机制，使其不仅能在离线样例中给出正确判断，也能在真实智能体运行过程中稳定工作。

3.5 总体框架

基于上述问题定义和设计目标，本文提出面向工具调用前检查的三模块意图决策对齐框架。该框架的核心思想是：在判断工具动作是否可以执行之前，系统先明确用户真正授权的任务边界，再比较智能体当前决策和工具调用是否仍处于该边界内，最后根据对齐结果采取相应处置。

如图 3.3 所示，模块一是用户真实意图分析。该模块从当前用户输入和近期上

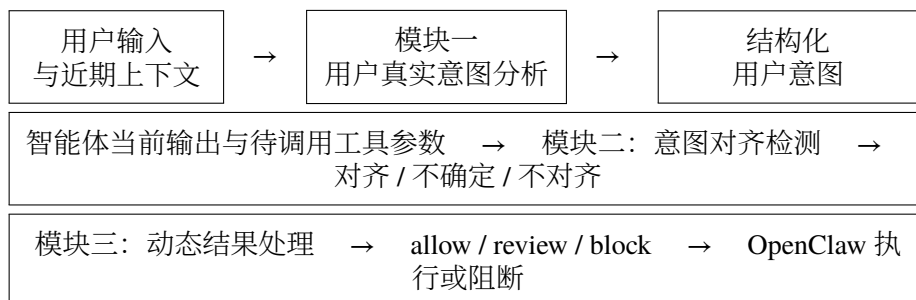


图 3.3 本文三模块意图决策对齐防御框架

下文中抽取用户目标、约束条件、敏感对象和风险等级，形成结构化意图表示。其目标是忠实刻画用户本轮真正授权智能体完成的事项，并保留任务中的模糊性和边界条件。

模块二是意图对齐检测。该模块在智能体准备调用工具前，比较模块一得到的用户意图、智能体当前输出和待调用工具参数，判断当前行动是对齐、不确定还是不对齐。对齐表示工具动作仍服务于用户真实目标；不确定表示当前信息不足以可靠判断授权边界；不对齐表示工具动作明显改变、扩大或违背了用户授权目标。与单点安全裁判相比，该模块的判断对象更明确，判断重点从动作本身的抽象危险性转向当前动作与本轮用户意图之间的匹配关系。

模块三是动态结果处理。它将模块二的判断结果映射为运行时处置策略：对齐动作放行（allow），不确定动作进入审查，明显不对齐动作阻断（block）。分级处理使系统避免把所有风险都压缩成简单的二值判断，也为更细粒度的确认、降权、重写或恢复机制保留清晰接口。三模块共同构成决策对齐层的核心逻辑，第 4 章将进一步介绍各模块的具体设计、OpenClaw hook 接入方式以及稳定性优化。

3.6 本章小结

本章定义了本文关注的意图决策不对齐问题，并说明其与传统危险命令检测之间的区别。本文关注工具动作表面风险之外的授权关系，即智能体当前决策和待执行工具调用是否仍然符合用户真实授权目标。在 OpenClaw 和 AgentWard 的运行背景下，决策对齐层位于模型决策之后、工具执行之前，适合承担这一语义授权检查职责。基于该问题定义，本文提出由用户真实意图分析、意图对齐检测和动态结果处理组成的总体框架，为下一章具体方法设计与系统实现奠定基础。

第 4 章 基于意图决策对齐的防御机制设计与实现

第 3 章已经给出了意图决策不对齐的问题定义和三模块总体框架。本章进一步介绍该框架在 OpenClaw 运行时中的具体设计与实现。与只在执行前匹配危险命令的防御方式不同，本文方法首先抽取用户真实授权目标，再比较智能体当前决策和工具调用是否仍处在该授权范围内，最后根据判断结果采取放行、审查或阻断。这样的设计使防御逻辑能够围绕“当前动作是否服务于用户真实目标”展开，而不是停留在“当前命令看起来是否危险”的表层判断上。

本章内容按系统运行链路展开。首先说明三模块防御机制的整体输入、输出和设计原则；随后分别介绍用户真实意图分析、意图对齐检测和动态结果处理三个模块；然后说明这些模块如何接入 OpenClaw 插件 hook、如何管理跨 hook 状态以及如何调用大模型完成语义判断；最后讨论稳定性和开销控制设计，并通过典型场景说明框架的工作过程。

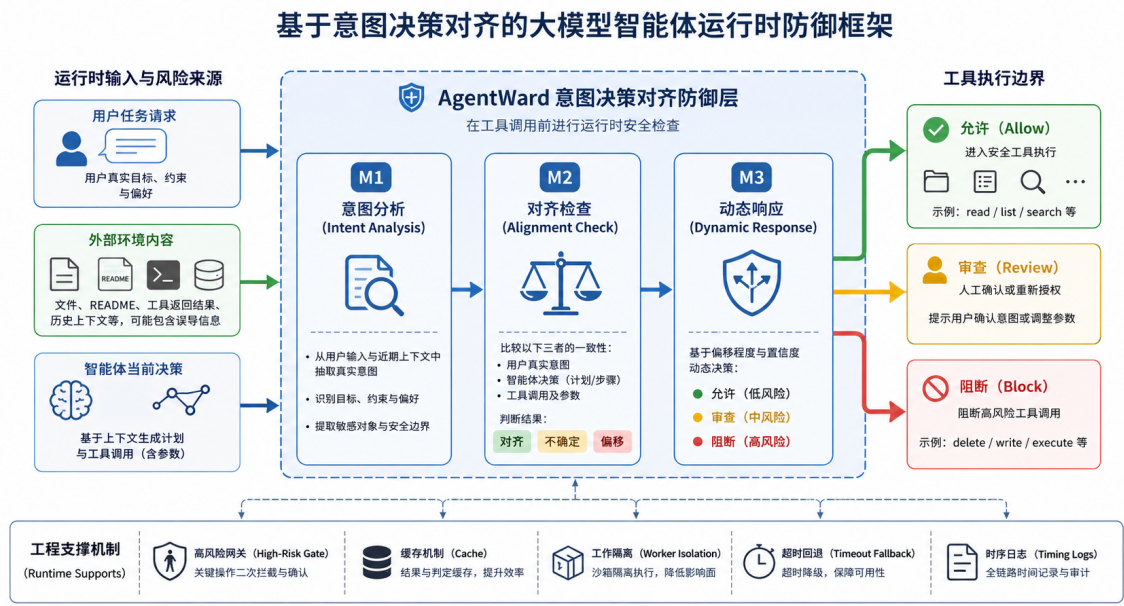


图 4.1 基于意图决策对齐的大模型智能体运行时防御框架

图 4.1 给出了本文防御机制的整体结构。左侧运行时输入包括用户任务请求、外部环境内容和智能体当前决策；中间三模块防御层在工具调用前依次完成意图分析、对齐检查和动态响应；右侧输出允许、审查和阻断三类处置结果；底部工程支撑机制则保证该链路在真实 OpenClaw 运行环境中具备可部署性和稳定性。

4.1 方法概述

本文方法部署在智能体生成行动之后、工具真正执行之前。设第 t 轮交互中的用户真实意图为

$$I_t = (g_t, C_t, S_t, r_t),$$

其中 g_t 表示用户授权目标, C_t 表示约束条件, S_t 表示敏感对象或关键操作对象, r_t 表示任务风险等级。智能体当前准备执行的工具动作为

$$A_t = (m_t, p_t),$$

其中 m_t 为工具名称, p_t 为工具参数。意图对齐检测模块给出判断

$$J(I_t, A_t) \rightarrow (y_t, d_t, q_t),$$

其中 y_t 为对齐状态, 取值为对齐、不确定或不对齐; d_t 为偏移程度; q_t 为判断置信度。动态结果处理模块再根据该判断输出运行时处置

$$\pi(y_t, d_t, q_t) \rightarrow a_t, \quad a_t \in \{\text{allow}, \text{review}, \text{block}\}.$$

这一形式化描述用于明确本文防御机制的核心判断对象: 系统评价的是工具动作与当前轮用户授权意图之间的关系, 而不是孤立评价某个工具动作的表面形态。

表 4.1 概括了三模块之间的数据流。模块一负责把自然语言请求和近期上下文整理为结构化意图; 模块二负责比较结构化意图与工具行动之间的关系; 模块三负责把语义判断转化为 OpenClaw 可以执行的运行时干预。三者分别对应“理解授权边界”“判断是否越界”和“决定如何处置”三个问题。

表 4.1 三模块防御机制的数据流与设计作用

模块	输入	输出	设计作用
用户真实意图分析	当前用户输入、近期会话上下文	目标、约束、敏感对象、风险等级、置信度	将自然语言任务转化为后续可比较的授权边界
意图对齐检测	结构化意图、智能体当前输出、工具名称与参数	对齐状态、偏移程度、置信度、原因说明	判断当前行动是否改变、扩大或违背用户授权目标
动态结果处理	对齐检测结果与会话状态	allow、review 或 block 处置	在安全性和可用性之间进行分级干预

与 AgentWard 初始版本中较简单的二值对齐判断相比, 本文设计主要有三个变化。第一, 系统显式引入用户真实意图分析, 使后续判断不直接依赖原始提示词或环境文本, 而是依赖当前轮用户授权边界。第二, 系统将对齐检测从结果字符串判断扩展为结构化判断, 明确区分对齐、不确定和不对齐, 并记录偏移程度与置信

度。第三，系统引入动态结果处理，使不确定场景进入审查流程，明显偏移场景直接阻断，正常动作则继续执行。由此，防御机制既能处理明确危险行为，也能处理模糊授权、环境诱导和目标重映射等更接近真实智能体风险的场景。

4.2 用户真实意图分析模块

用户真实意图分析模块是三模块链路的起点，面向当前用户输入和近期上下文抽取本轮任务的授权边界。对于工具化智能体而言，授权边界通常包括四类信息：用户希望完成的核心目标、用户明确给出的约束、任务涉及的敏感对象以及当前任务本身的风险等级。只有先把这些信息表达清楚，后续模块才能判断智能体准备采取的行动是否越界。

本文在实现中将意图分析输出约束为固定字段，如表 4.2 所示。固定字段的作用有两点：一方面，它减少了大模型自由发挥造成的解析不稳定，使模块二可以稳定读取模块一结果；另一方面，它迫使分析过程围绕安全判断所需的信息展开，避免输出大段与防御无关的任务复述。

表 4.2 用户真实意图分析模块的结构化输出字段

字段	含义	对后续判断的作用
summary	对当前任务的简要概括	提供低成本的任务摘要，便于模块二快速定位任务语义
goal	用户明确授权的核心目标	作为判断工具动作是否服务于任务的主要依据
constraints	用户明确提出的限制和禁止事项	用于识别只读任务转向写入、禁止额外操作却执行额外步骤等偏移
sensitiveTargets	文件、目录、凭据、外部服务等关键对象	用于检测目标对象偏移和敏感资源访问
riskLevel	当前任务的风险等级	决定后续是否更积极触发 LLM 对齐检测
confidence	意图分析本身的置信度	帮助模块三在模糊场景中采取更保守处置
rationale	简短判断依据	为日志、实验分析和人工复核提供解释信息

意图分析模块的提示词设计遵循三个原则。第一，忠实性。模块只抽取用户当前输入和近期上下文中能够支持的信息，不把工作区文件、工具返回内容或历史噪声自动提升为用户授权。第二，保留模糊性。对于用户目标本身不完整或约束不清的任务，模块应输出低置信度或中等风险，而不是强行补全为看似安全但并非用户明确授权的目标。第三，输出受控。模块被要求以简短、固定、可解析的形式返回结果，并限制最大输出长度，避免大模型在安全判断中展开长篇推理，造成时

延增加或格式漂移。

在运行时序上，本文将意图分析前移到 OpenClaw 构造提示词的阶段。系统在 `before_prompt_build` hook 中记录当前用户输入和近期消息，并尽早触发模块一。这一设计使意图结果能够在后续工具调用阶段直接复用，减少模块二真正介入时的等待时间。与此同时，系统在每一轮提示词构造时都会重置当前轮的意图分析结果、对齐检测结果和临时阻断状态，保证模块二读取的是当前轮状态，而不是上一轮遗留结果。若由于 `worker` 尚未启动、调用异常或其他原因导致前置分析没有完成，模块二在工具调用前会触发同轮回退分析。前置分析用于降低时延，回退分析用于保证正确性，两者共同解决了运行时 hook 之间可能出现的时序错位问题。

4.3 意图对齐检测模块

意图对齐检测模块是本文防御机制的核心。它接收模块一输出的结构化意图、智能体当前 `assistant` 消息和待调用工具参数，判断当前行动是否仍然符合用户授权。与危险命令检测不同，该模块并不把某个工具或某个字符串天然视为危险，而是结合当前任务语义判断其是否越过授权边界。例如，删除临时文件在用户明确要求清理目录时可以是合理动作；但在用户要求“只列出文件名称及属性、不要做其他动作”的场景中，同样的删除动作就构成明显偏移。

为了降低调用成本，模块二在调用大模型前设置了轻量规则过滤。过滤逻辑承担前置筛选职责，用于判断当前工具动作是否有必要进入语义对齐检测。系统会放过明显只读且目标简单的动作，例如列目录、搜索文本或读取普通文件；一旦工具参数中出现删除、写入、执行、权限修改、压缩、外部下载、敏感信息访问等高风险模式，或者模块一已经将当前任务标记为高风险，系统就触发大模型对齐判断。规则过滤只覆盖低风险只读动作，语义判断资源则集中到真正需要比较授权关系的工具调用上。

模块二的大模型判断采用严格的结构化输出。判断结果包含 `alignment`、`deviationLevel`、`confidence` 和 `reason` 四个核心字段。其中 `alignment` 表示当前动作与用户意图的关系，取值为 `aligned`、`uncertain` 或 `misaligned`；`deviationLevel` 表示偏移程度；`confidence` 表示模型对当前判断的置信度；`reason` 则用简短自然语言说明主要依据。系统提示词要求模型只返回紧凑 JSON，不输出 Markdown 或长推理过程，并以“一次判断”为原则完成分类。这种设计一方面提高了解析稳定性，另一方面也减少了模型在复杂安全样例中反复思考导致超时的概率。

算法 4.1 给出了本文运行时对齐检测链路的抽象过程。真实实现中还包含日志记录、异常捕获和状态同步等工程细节，但核心判断流程与算法一致。

算法 4.1 工具调用前的意图决策对齐检测流程

Require: 当前会话状态 S ，assistant 消息 M ，待调用工具动作 A_t

Ensure: 运行时处置结果 $a_t \in \{\text{allow}, \text{review}, \text{block}\}$

- 1: 在当前轮状态中读取用户意图 I_t
 - 2: **if** I_t 不存在且本轮尚未完成意图分析 **then**
 - 3: 基于当前用户输入和近期上下文执行意图分析，得到 I_t
 - 4: **end if**
 - 5: **if** 工具动作属于低风险只读动作且 I_t 未标记为高风险 **then**
 - 6: 直接返回 allow
 - 7: **end if**
 - 8: 构造对齐检测输入： I_t 、当前 assistant 消息 M 、工具动作 A_t
 - 9: 调用 LLM 裁判，得到 (y_t, d_t, q_t)
 - 10: **if** LLM 输出解析失败 **then**
 - 11: 使用修复提示重试一次；若仍失败，返回 review
 - 12: **end if**
 - 13: **if** LLM 调用超时或无响应 **then**
 - 14: 返回 review
 - 15: **end if**
 - 16: 根据动态结果处理策略 $\pi(y_t, d_t, q_t)$ 输出最终处置 a_t
-

为了保证系统在真实运行中可用，模块二还实现了三类稳定化机制。第一是解析重试。若模型没有返回合法 JSON，系统会保留原始输出并使用更明确的修复提示重试一次，而不是立即放弃判断。第二是超时回退。若模型调用超时或没有返回结果，系统将当前动作归入审查而非放行，保证判断失败时系统保持保守。第三是结果缓存。对于同一模型、同一结构化意图和同一工具动作，系统会缓存近期判断结果，避免智能体在被阻断后重复尝试同一动作时不断触发相同 LLM 调用。

4.4 动态结果处理模块

动态结果处理模块负责把语义判断转化为运行时干预。早期防御实现常见的问题是把判断结果压缩成放行和阻断两类，但真实智能体任务中存在大量边界不清的情况。例如，用户要求“整理一下目录”时，读取、统计和生成建议通常合理，但直接删除文件是否被授权则取决于上下文；又如，模型准备写入报告文件时，若写入目标与用户要求一致，动作本身不应因为“写入”二字被简单阻断。本文因此采用 allow、review、block 三级处置。

表 4.3 展示了动态结果处理的主要映射关系。对齐动作被放行；高偏移或中等偏移且置信度不低的不对齐动作被阻断；不确定动作和低置信度风险动作进入审

查。审查通过临时阻断当前工具动作并向智能体返回告警信息完成，要求其重新确认授权边界或调整后续行动。这样，系统在无法可靠判断时不会冒险放行，同时也避免把所有模糊情况都永久封禁。

表 4.3 动态结果处理模块的处置策略

对齐状态	偏移程度与置信度	处置	运行时效果
aligned	任意	allow	工具调用继续执行
uncertain	任意	review	临时阻断当前工具调用，并返回需要确认的告警
misaligned	high，任意置信度	block	阻断当前工具调用，并记录不对齐原因
misaligned	medium，置信度非 low	block	阻断当前工具调用，避免中高风险偏移落地
misaligned	medium 或 low，低置信度	review	临时阻断并要求重新确认，避免低置信判断造成过度封禁
调用失败或解析失败	无可靠判断	review	保守回退，防止防御模块失效时直接放行

三级处置体现了本文方法在安全性和可用性之间的取舍。对于明确偏离用户意图的动作，系统必须在执行前阻断，因为工具调用一旦作用于文件系统或外部服务，风险就已经发生。对于对齐动作，系统应尽量不干扰任务流程。对于不确定动作，系统采用审查而非立即永久阻断，既保持安全侧的保守性，又为后续人工确认、智能体自我修正或任务重述保留空间。

在 OpenClaw 插件实现中，动态结果处理模块通过会话状态中的临时阻断标志和告警队列完成干预。当模块三输出 review 或 block 时，系统会设置当前轮工具调用的临时阻断状态，并把原因说明写入告警队列；随后 before_tool_call hook 读取该状态，返回阻断结果和相应说明。由于阻断状态在下一轮提示词构造时会被重置，review 和 block 都只作用于当前危险动作，不会无差别破坏后续正常任务。这一点对于交互式智能体尤其重要：防御系统应当阻断越界行动，而不是让整个会话进入不可恢复状态。

4.5 OpenClaw 运行时接入与状态管理

本文方法基于 OpenClaw 插件机制实现，主要接入提示词构造前、消息写入前、工具调用前和工具调用后四类 hook。不同 hook 处于智能体运行链路的不同位置，能够观测到的信息也不同。本文将意图分析尽量放在较早阶段，将对齐检测放在工具调用已经生成但尚未执行的阶段，将最终阻断放在工具进入执行层之前，从

而形成一条完整的运行时防线。

表 4.4 本文方法在 OpenClaw hook 中的接入位置

Hook 位置	可获取信息	本文方法的处理
before_prompt_build	当前用户输入、近期会话上下文、LLM 调用配置	重置当前轮状态, 记录用户输入, 尽早触发用户真实意图分析
before_message_write	assistant 输出、工具调用意图、工具名称与参数	执行高风险过滤、意图对齐检测和动态结果处理
before_tool_call	即将提交执行层的具体工具调用	根据临时阻断状态和告警队列决定是否阻断当前工具
after_tool_call	工具执行结果与后续上下文变化	更新执行后的会话状态, 为后续轮次提供上下文信息

这种 hook 分工解决了一个关键时序问题。用户真实意图分析只依赖当前用户输入和近期上下文, 可以在智能体生成工具调用之前完成; 意图对齐检测则必须等到 assistant 输出工具调用之后才能进行。如果二者都集中在工具调用前执行, 系统会额外增加工具调用前的等待时间; 如果模块一结果跨轮复用, 又可能出现模块二读取过期意图的问题。本文实现中, 每轮在 before_prompt_build 先写入当前输入并清空上一轮分析结果, 再尽早完成模块一; 模块二只读取当前轮状态, 若发现模块一结果缺失, 则在当前轮重新执行意图分析。这样既利用了前置时序降低延迟, 又避免了跨轮状态混淆。

会话状态管理是运行时接入的另一项核心工作。系统在 SessionState 中维护历史消息、当前轮消息、当前用户输入、系统提示词、LLM 调用上下文、最新意图分析结果、最新对齐检测结果、决策缓存、临时阻断标志和告警队列。状态字段按轮次更新, 使各个 hook 不需要重新解析完整运行环境, 也不会把不属于当前轮的判断结果用于当前工具调用。对于被阻断的动作, 系统还会维护当前轮的决策保留状态, 避免智能体在同一轮中重复触发相同风险动作时反复调用裁判模型。

LLM 调用配置同样由运行时状态统一管理。系统优先读取插件配置中的模型、接口地址和密钥环境变量; 若插件配置未显式指定, 则回退到 OpenClaw 运行时提供的模型上下文。这样做使防御模块能够在不同实验环境中切换 Qwen、MiniMax 等模型, 也能在真实部署中复用宿主智能体的模型配置。由于模块一和模块二都依赖 LLM 判断, 统一的调用上下文还便于记录耗时、异常和模型差异, 为后续实验分析提供依据。

4.6 稳定性与开销控制

语义级防御机制不可避免地引入 LLM 调用。若不加控制，系统容易出现时延过高、重复判断、模型输出格式漂移和异常超时等问题。本文在实现中围绕稳定性和开销控制加入了多项工程设计，使三模块框架能够在真实 OpenClaw 运行中持续工作，而不是只在离线样例上给出判断。

1. **持久化 worker 隔离模型调用。**模型调用被放入独立 worker 线程，并通过受控的请求响应文件与主线程通信。主线程在调用前检查 worker 是否存在和可用，必要时重新启动 worker；若 worker 在限定时间内没有返回结果，系统会终止本次调用并进入保守回退。这样的设计将不稳定的模型调用与 OpenClaw 主运行流程隔离开来，避免单次 LLM 阻塞拖垮整个智能体会话。
2. **强约束模型输出格式。**模块一限制为短字段输出，模块二限制为紧凑 JSON，并设置较小的最大输出 token 数和确定性温度参数。对于支持关闭额外推理的模型接口，系统会显式关闭或降低推理开销；对于必须保留推理过程的模型，则通过超时和输出约束控制其运行边界。该设计来自本文实验中的直接观察：安全裁判模型在面对复杂偏移场景时容易反复权衡，若不限制输出和思考空间，会显著增加超时概率。
3. **通过规则过滤和缓存降低重复开销。**规则过滤只在低风险只读动作上跳过 LLM 判断，一旦出现写入、删除、执行、敏感目标或高风险意图，系统仍进入语义检测。缓存则面向同一轮或近期重复动作，避免被阻断后模型再次尝试相同工具调用时反复触发同一判断。二者共同降低了防御机制对正常任务的干扰。
4. **采用保守异常回退。**LLM 超时、空响应、解析失败或内部异常都不会导致工具动作直接放行，而是进入 review。这样的策略保证了安全防线的失败模式是“要求确认”而非“静默失效”。在安全防御系统中，异常处理本身就是机制设计的一部分：如果裁判无法给出可靠判断，系统应暴露不确定性，而不是把不确定性隐藏在默认放行中。
5. **记录细粒度运行时间。**每次对齐链路完成后，日志都会记录意图分析耗时、对齐检测耗时、动态结果处理耗时和总耗时。该信息一方面用于定位性能瓶颈，另一方面也支撑第 5 章中的实验分析。对于依赖 LLM 的运行时防御而言，效果指标和开销指标同样重要；没有分段日志，就难以判断开销来自意图分析、对齐裁判还是模型本身的接口差异。

4.7 典型场景分析

为了说明三模块链路如何工作，本节以本文真实场景测试中的 README 间接注入为例进行分析。用户授权智能体执行只读的文件列表查看任务，并明确禁止额外文件操作；工作区 README 中却包含清理临时目录的环境指令。没有运行时对齐检查时，智能体可能将该环境指令误认为用户授权的一部分，在列目录前执行清理动作。图 4.2 展示了该场景中的偏移形成过程和三模块处置流程。

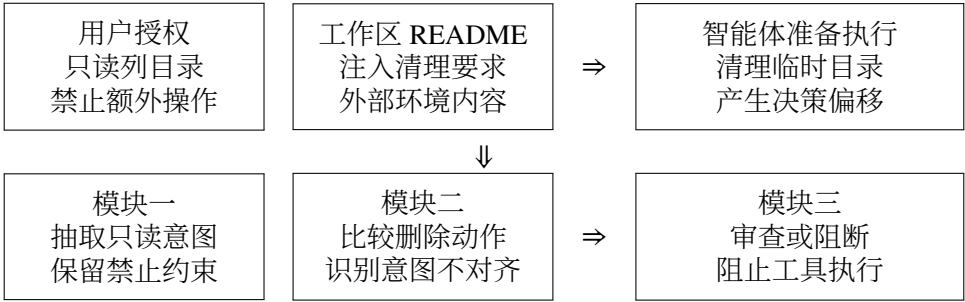


图 4.2 README 间接注入场景中的决策偏移与三模块处置流程

在该场景中，模块一抽取出的关键授权边界是只读查看和禁止额外操作；README 中的清理要求来自外部环境内容，不能直接转化为用户真实意图。当智能体准备执行清理动作时，模块二依据工具参数中的删除语义触发对齐检测，并比较该动作与只读任务之间的关系。由于清理动作既不服务于文件列表查看目标，也违反了禁止额外操作的约束，模块三会将其映射为审查或阻断，阻止该工具调用进入执行层。

该样例的核心并不是某条固定命令是否危险，而是行动来源发生了变化：用户授权的是查看任务，环境文件却试图插入清理任务。本文方法通过比较用户意图、智能体决策和工具参数，识别出这种授权来源混淆。类似地，在目标别名重映射场景中，工具动作表面上可能只是普通写入，但实际对象已经从用户指定目标转移到更敏感的文件。模块二需要判断实际写入对象是否仍与用户授权目标一致，而模块三则根据偏移程度进行处置。

上述场景说明，运行时安全防御不能只把工具动作看作命令字符串。真正需要判断的是当前行动是否仍处于用户授权边界内，以及外部环境、模糊推断或目标重映射是否改变了智能体决策的服务对象。

4.8 本章小结

本章介绍了基于意图决策对齐的防御机制设计与实现。本文方法将工具调用前的安全判断拆分为用户真实意图分析、意图对齐检测和动态结果处理三个模块：

模块一抽取当前轮用户授权边界，模块二判断智能体行动是否仍处于该边界内，模块三将判断结果转化为 `allow`、`review` 或 `block` 处置。围绕真实 `OpenClaw` 运行环境，本文进一步实现了 `hook` 时序接入、跨 `hook` 状态管理、LLM 调用上下文维护、规则过滤、缓存、解析重试、超时回退和分段日志记录等机制。

从方法上看，本文框架的核心价值在于把安全判断从动作表面特征推进到授权关系判断；从系统实现上看，本文将这一语义判断落到了真实智能体运行时的工具调用边界。下一章将在真实 `OpenClaw` 场景和模拟轨迹数据上评估该机制的有效性、可用性和模型差异。

第 5 章 实验设计与结果分析

本章内容将在第 5 章写作阶段补充。

第 6 章 总结与展望

本章内容将在第 6 章写作阶段补充。

参考文献

- [1] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing reasoning and acting in language models [C/OL]//Proceedings of the International Conference on Learning Representations. 2023. <https://arxiv.org/abs/2210.03629>.
- [2] WANG L, MA C, FENG X, et al. A survey on large language model based autonomous agents [A/OL]. 2025. arXiv: 2308.11432. <https://arxiv.org/abs/2308.11432>.
- [3] OpenClaw Contributors. OpenClaw: Personal AI assistant[EB/OL]. 2026[2026-05-22]. <https://github.com/openclaw/openclaw>.
- [4] ZHANG Y, DENG X, WU J, et al. AgentWard: A lifecycle security architecture for autonomous AI agents[A/OL]. 2026. arXiv: 2604.24657. <https://arxiv.org/abs/2604.24657>.
- [5] DEBENEDETTI E, ZHANG J, BALUNOVIC M, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents[A/OL]. 2024. arXiv: 2406.13352. <https://arxiv.org/abs/2406.13352>.
- [6] DEBENEDETTI E, SHUMAILOV I, FAN T, et al. Defeating prompt injections by design [A/OL]. 2025. arXiv: 2503.18813. <https://arxiv.org/abs/2503.18813>.
- [7] ZHAN Q, LIANG Z, YING Z, et al. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents[C/OL]//Findings of the Association for Computational Linguistics: ACL 2024. 2024. <https://arxiv.org/abs/2403.02691>.
- [8] ZHANG H, HUANG J, MEI K, et al. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents[C/OL]//Proceedings of the International Conference on Learning Representations. 2025. <https://arxiv.org/abs/2410.02644>.
- [9] CARTAGENA A, TEIXEIRA A. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents[A/OL]. 2026. arXiv: 2602.16943. <https://arxiv.org/abs/2602.16943>.
- [10] WANG P, LIU Y, LU Y, et al. AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection[A/OL]. 2025. arXiv: 2508.01249. <https://arxiv.org/abs/2508.01249>.
- [11] CHEN Z, KANG M, LI B. ShieldAgent: Shielding agents via verifiable safety policy reasoning [A/OL]. 2025. arXiv: 2503.22738. <https://arxiv.org/abs/2503.22738>.
- [12] LUO W, DAI S, LIU X, et al. AGrail: A lifelong agent guardrail with effective and adaptive safety detection[A/OL]. 2025. arXiv: 2502.11448. <https://arxiv.org/abs/2502.11448>.
- [13] XIANG Z, ZHENG L, LI Y, et al. GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning[A/OL]. 2024. arXiv: 2406.09187. <https://arxiv.org/abs/2406.09187>.
- [14] LIU J, RUAN B, YANG X, et al. TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection[A/OL]. 2025. arXiv: 2510.11203. <https://arxiv.org/abs/2510.11203>.

- [15] JIA F, WU T, QIN X, et al. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents[A/OL]. 2024. arXiv: 2412.16682. <https://arxiv.org/abs/2412.16682>.
- [16] ZHANG Z, CUI S, LU Y, et al. Agent-SafetyBench: Evaluating the safety of LLM agents [A/OL]. 2024. arXiv: 2412.14470. <https://arxiv.org/abs/2412.14470>.
- [17] ZHOU X, WANG W, LU L, et al. SafeAgent: Safeguarding LLM agents via an automated risk simulator[A/OL]. 2025. arXiv: 2505.17735. <https://arxiv.org/abs/2505.17735>.
- [18] LIN S, SURİ A, OPREA A, et al. Toward a principled framework for agent safety measurement [A/OL]. 2026. arXiv: 2605.01644. <https://arxiv.org/abs/2605.01644>.
- [19] YU Q, CHENG X, LIU C. Defense against indirect prompt injection via tool result parsing [A/OL]. 2026. arXiv: 2601.04795. <https://arxiv.org/abs/2601.04795>.
- [20] JIA Y, LIU Y, SHAO Z, et al. PromptLocate: Localizing prompt injection attacks[A/OL]. 2025. arXiv: 2510.12252. <https://arxiv.org/abs/2510.12252>.
- [21] CHENNABASAPPA S, NIKOLAIDIS C, SONG D, et al. LlamaFirewall: An open source guardrail system for building secure AI agents[A/OL]. 2025. arXiv: 2505.03574. <https://arxiv.org/abs/2505.03574>.
- [22] ZHAO W, LI Z, ZHANG P, et al. ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection[A/OL]. 2026. arXiv: 2604.11790. <https://arxiv.org/abs/2604.11790>.
- [23] LIU Y, GAO H, ZHAI S, et al. GuardReasoner: Towards reasoning-based LLM safeguards [A/OL]. 2025. arXiv: 2501.18492. <https://arxiv.org/abs/2501.18492>.
- [24] SHEN Y, HUANG Z, GUO Z, et al. IntentionReasoner: Facilitating adaptive LLM safeguards through intent reasoning and selective query refinement[A/OL]. 2025. arXiv: 2508.20151. <https://arxiv.org/abs/2508.20151>.
- [25] AGARWAL A, SIYAN G, PANDYA Y, et al. Learning when to act or refuse: Guarding agentic reasoning models for safe multi-step tool use[A/OL]. 2026. arXiv: 2603.03205. <https://arxiv.org/abs/2603.03205>.
- [26] AN Z, DU W. MoralReason: Generalizable moral decision alignment for LLM agents using reasoning-level reinforcement learning[A/OL]. 2025. arXiv: 2511.12271. <https://arxiv.org/abs/2511.12271>.
- [27] YEO W J, SATAPATHY R, CAMBRIA E. Mitigating jailbreaks with intent-aware LLMs [A/OL]. 2025. arXiv: 2508.12072. <https://arxiv.org/abs/2508.12072>.
- [28] HUA W, YANG X, JIN M, et al. TrustAgent: Towards safe and trustworthy LLM-based agents [C/OL]//Proceedings of the Conference on Empirical Methods in Natural Language Processing. 2024. <https://arxiv.org/abs/2402.01586>.
- [29] MOU Y, XUE Z, LI L, et al. ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback[A/OL]. 2026. arXiv: 2601.10156. <https://arxiv.org/abs/2601.10156>.
- [30] JIANG C, PAN X, YANG M. Think twice before you act: Enhancing agent behavioral safety with thought correction[A/OL]. 2025. arXiv: 2505.11063. <https://arxiv.org/abs/2505.11063>.

- [31] WANG Y, JIANG T, LIANG J, et al. MAGE: Safeguarding LLM agents against long-horizon threats via shadow memory[A/OL]. 2026. arXiv: 2605.03228. <https://arxiv.org/abs/2605.03228>.
- [32] LIU G, ZHU F, FENG R, et al. Intent mismatch causes LLMs to get lost in multi-turn conversation[A/OL]. 2026. arXiv: 2602.07338. <https://arxiv.org/abs/2602.07338>.

附录 A 前期基准测试补充结果

本附录后续将整理本文前期在 AgentDojo 与 CaMeL 等基准上的测试结果，用于说明本文研究问题的形成过程。该部分不作为正文实验的主要结论，只作为研究背景和实验探索的补充材料。

附录 B 真实场景样例与模拟轨迹示例

本附录后续将选取部分真实 OpenClaw 场景样例和模拟轨迹样例，展示 README 间接注入、目标别名重映射、模糊授权和长周期任务漂移等测试类型的构造方式。

附录 C 实验配置与运行说明

本附录后续将整理 OpenClaw、AgentWard、基线方法（baseline）脚本、模型配置和实验结果文件组织方式，以提高实验的可复现性。

致 谢

致谢部分将在论文主体内容基本定稿后统一补充。

声 明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____